2026 ICEENG International Conference for Innovations in Intelligent Computing and Cybersecurity (IICC) | 979-8-3315-4702-8/26/$31.00 ©2026 IEEE | DOI: 10.1109/IICC69623.2026.11582734

Multi-Agent Racing Trajectory Optimization with Tire Wear Awareness: A Comparison of Reinforcement Learning and Metaheuristic methods

Farida Gamal*, Sara Alajmy§, Salma Elhabashy§, Ziad ElGendy§,
Farah Eltaher§, Omar Ayoub§, Mohamed A. Ibrahim*, and Omar M. Shehata*
*Mechatronics Engineering Department, German University in Cairo, Cairo, Egypt
§Media Engineering Technology Department, German University in Cairo, Cairo, Egypt
Email: {farida.saleh, sara.alajmy, salma.elhabashy, ziad.elgendy, farah.eltaher, omar.ayoub}@student.guc.edu.eg,
{mohamed.salahelden, omar.mohamad}@guc.edu.eg

***Abstract*—This paper implements multiple algorithms to achieve racing line optimization with respect to tire wear and lap time minimization on a 3 DOF model-based vehicle and Pacejka tire model. We evaluated metaheuristic algorithms such as Simulated Annealing (SA), Genetic algorithms (GA), Particle Swarm Optimization (PSO), and Harris Hawks Optimization (HHO) and compared their results to Machine Learning approaches such as Reinforcement Learning (PPO). The primary testing ground was the Autodromo Nazionale Monza which acted as benchmark to obtained results. We evaluated each method across three key performance indicators lap time, tire degradation, and computational latency. While population-based metaheuristics achieved better lap time results (76.17s), they are computationally expensive (> 866.11s runtime). However, the RL agent demonstrated a reduction in computational time (14.13s), suggesting real-time generation with only a marginal 3.5% deficit in lap time performance (78.87s). These results highlight that while meta-heuristic algorithms achieve better overall results their extreme computational need makes the RL results better.**

***Index Terms*—Reinforcement Learning, Trajectory Optimization, PPO, Metaheuristics, Racing**

I. Introduction

The optimal racing line is critical in affecting racing performance. Multiple aspects influence the optimal racing line choice.

Recent advancements in racing line optimization have shifted towards data-driven and computationally efficient frameworks to overcome the limitations of traditional hand-tuned methods. Jain et al. [1] proposed a fully data-driven method that derives minimum-time racing lines using Bayesian optimization. By utilizing waypoint coordinates, track width, and measurable vehicle parameters to guide the sampling of smooth trajectories, this approach accelerates the pre-computation of racing lines for autonomous racing teams. However, this was done on a single vehicle. The main focus was to achieve minimum time but do not take tire wear into consideration.

Moreover, researchers have increasingly turned to meta-heuristic algorithms to handle high dimensionality and non-linearity. Negură et al. [2] formulate lap-time minimization as an optimal control problem, applying Genetic Algorithms (GA), Particle Swarm Optimization (PSO), and Simulated Annealing (SA) to a vehicle model that accounts for aerodynamics and grip limits. While this study addresses the gaps found in existing methods regarding non-linearity and uncertainty, it notes that the omission of real-world factors such as dynamic tire wear remains a limitation for achieving high-fidelity simulated improvements. Moreover all tests were done in optimizing a single vehicle's racing line without accounting for other vehicles.

Aly et al. [3] extend the Coronavirus Optimization Algorithm (CVOA) to minimize both lap time and energy consumption for a BMW i3 model. When compared against GA and Grey Wolf Optimization (GWO) on small-scale tracks, this method effectively addresses the need for scalable, fast offline optimization. However, the study highlights unresolved issues, such as the absence of steering-angle constraints, a lack of testing on larger track layouts, and the need for comparison against modern machine-learning techniques.

Existing approaches struggle with the non-linearity, high dimensionality, and uncertainty of dynamic racing environments [4]. Traditional optimization in this domain relies on simplifying assumptions that may not hold in real-world racing. A significant limitation is that these studies omit several real-world factors, including tire wear, fuel load, and driver behavior, limiting the simulated lap-time results.

Chen et al. [5] enhances the standard minimum-curvature method by incorporating track kerbs and a late-apex technique, achieving a 90% increase in accuracy. While this addresses the historical neglect of kerbs and driver strategies, the authors find that the late-apex modification did not yield better lap times and note that the omission of complex vehicle dynamics such as temperature and aerodynamics requires further evaluation with richer models.

This paper presents four meta-heuristic algorithms (SA, GA, PSO, and HHO) against Reinforcement Learning (PPO) for multi-agent racing line optimization. By integrating a physics-based tire wear model within a 3DOF single track vehicle dynamics model, we evaluate trajectories for lap time and tire wear minimization. Our analysis identifies the performance trade-offs between these algorithms.

II. Vehicle Dynamics Model

To capture the essential vehicle behavior for trajectory optimization, a 3-DOF single-track dynamic model was used alongside a simplified Magic Formula tire model . The model formulation is adopted from Rajamani [6]. The state vector is defined as $\mathbf{x} = [v_x, v_y, r, X, Y, \psi, w]^T$, representing longitudinal velocity, lateral velocity, yaw rate, global position (X, Y), heading angle, and the tire friction state of health (SOH), respectively.

A. Vehicle Model

The rigid body dynamics are governed by the Newton-Euler equations of motion. Aerodynamic drag (F_{drag}) and rolling resistance (F_{roll}) are modeled using lumped constant coefficients [7]:

$$\dot{v}_x = \frac{1}{m}\left(F_x - F_{y,f}\sin(\delta) - F_{drag} - F_{roll} + m v_y r\right) \quad (1)$$

$$\dot{v}_y = \frac{1}{m}\left(F_{y,r} + F_{y,f}\cos(\delta) - m v_x r\right) \quad (2)$$

$$\dot{r} = \frac{1}{I_z}\left(l_f F_{y,f}\cos(\delta) - l_r F_{y,r}\right) \quad (3)$$

where m is the vehicle mass and I_z is the yaw moment of inertia.

B. Tire Model

The lateral forces for the front ($F_{y,f}$) and rear ($F_{y,r}$) tires are generated using a modified Pacejka Magic Formula [8]. The peak friction coefficient is dynamically scaled by the wear state $w(t)$:

$$F_{y,i}(\alpha_i, w) = w(t)\cdot D_i \sin\Big(C_i \arctan\big(B_i\alpha_i - E_i(B_i\alpha_i - \arctan(B_i\alpha_i))\big)\Big) \quad (4)$$

for $i \in \{f, r\}$. The slip angles α_f and α_r are derived from the vehicle velocities and steering input δ:

$$\alpha_f = \arctan\left(\frac{v_y + l_f r}{v_x}\right) - \delta, \quad \alpha_r = \arctan\left(\frac{v_y - l_r r}{v_x}\right) \quad (5)$$

For this study, aerodynamic downforce is neglected to isolate mechanical grip characteristics. The vertical tire loads are approximated using the static weight distribution:

$$F_{z,f} = \frac{l_r}{L} m g, \quad F_{z,r} = \frac{l_f}{L} m g \quad (6)$$

where $L = l_f + l_r$ is the vehicle wheelbase.

Tire wear is a complex path-dependent process. While higher accuracy models account for thermal states and frictional power dissipation [9], these formulations introduce significant computational complexity.

To minimize computational cost, we used a simplified model grounded in Archard's theory of wear [10], which establishes that the wear rate is linearly proportional to the applied load. Adapting this principle to vehicle dynamics assuming the dominant stressor during limit cornering is the lateral load we model the degradation rate $\dot{w}$ as proportional to the total lateral force normalized by the vehicle weight:

$$\dot{w} = -\kappa_{wear}\frac{|F_{y,f}| + |F_{y,r}|}{F_z} \quad (7)$$

where $\kappa_{wear} = 0.001$ is the empirical wear rate coefficient and $w(t)$ is initialized at $w(0) = 1$ (100% health). This linear formulation captures the fundamental trade-off between cornering aggressiveness and tire preservation while avoiding additional nonlinearity in the optimization cost.

C. Constraints

1) **Track Limits:** $-w_{inner}(s) \le d_{lateral}(X, Y, s) \le w_{outer}(s)$.
2) **Velocity Limits:** $0 < v_x \le 350$ km/h.
3) **Steering Limits:** $-50^\circ < \delta < 50^\circ$.

where w_{inner} and w_{outer} denote the distance from the centerline to the inner and outer track edges, respectively.

III. Metaheuristic Approaches

A. Simulated Annealing (SA)

To establish a baseline for single-trajectory optimization, we implemented a Simulated Annealing solver [11]. SA operates as a probabilistic local search, iteratively perturbing a candidate racing line to escape local optima.

1) Initialization: The algorithm initializes with the track centerline. At each iteration k, a neighbor state $\mathbf{X}_{new}$ is generated by applying Gaussian noise to the current lateral offsets $\mathbf{X}_{curr}$:

$$\mathbf{X}_{new} = \text{clip}(\mathbf{X}_{curr} + \mathcal{N}(0, \sigma_d^2), -d_{max}, d_{max}) \quad (8)$$

where the perturbation scale $\sigma_d = 0.1$ ensures that changes are small enough to maintain drivability but large enough to explore the lane width within a range $d_{max} = 1.5$.

2) Cooling: To prevent premature convergence, we employ a Metropolis loop with $N_{inner} = 3$ iterations at each temperature level. A candidate solution with a higher cost ($\Delta J > 0$) is accepted with probability:

$$P_{accept} = \exp\left(-\frac{\Delta J}{T_k}\right) \quad (9)$$

The system cools according to a geometric schedule $T_{k+1} = \alpha_{SA} T_k$ with a decay factor $\alpha_{SA} = 0.95$. This schedule allows the solver to explore the global solution space aggressively in early iterations (high T) before settling into an exploitation (low T). The algorithm terminates when it reaches a temperature of $T_{final} = 1$ or a maximum limit of $N_{max} = 100$ iterations.

TABLE I: Simulated Annealing Parameters

Parameter	Value
Initial Temperature (T_{init})	100
Cooling Factor (α_{SA})	0.95
Max Iterations	100
Inner Iterations	3

B. Genetic Algorithm (GA)

We implement a Genetic Algorithm [12] to perform a global search of the solution space. The GA evolves a population of $N_{pop} = 20$ racing lines over $N_{gen} = 100$ generations, making it less likely to get trapped in local optima than it is when using a gradient-based method.

1) Chromosome Encoding and Initialization: Each individual is represented by a matrix of lateral offsets $\mathbf{O} \in \mathbb{R}^{N_{points} \times 2}$ where $N_{points} = 5$. The initial population is composed of:

- **Lane Heuristics (30%):** Deterministic combinations of extreme lines such as Vehicle 1 on Left Limit and Vehicle 2 on Right Limit to ensure the search space boundaries are covered.
- **Stochastic Smooth Heuristics (70%):** Random lateral offsets smoothed via a Gaussian filter to ensure the initial trajectories are physically drivable.

2) Evolutionary Operators: The population evolves through a 20/60/20 split to balance exploration and exploitation:

1) **Elitism (20%):** The top 20% of individuals with the lowest cost J are preserved unchanged to ensure consistent performance improvement.
2) **Crossover (60%):** Parents are chosen via **Tournament Selection** ($k = 2$), where two individuals are chosen randomly, and the fitter one is selected. We utilize an Arithmetic Crossover with a factor of $\alpha_{GA} = 0.7$. To prevent the generation of unrealistic paths, a Gaussian smoothing filter is applied.
3) **Mutation (20%):** Selected parents undergo mutation where Gaussian noise ($\sigma_{GA} = 0.5$) is added to the trajectory.

TABLE II: Genetic Algorithm Configuration

Parameter	Value
Population Size	20
Generations	100
Selection Method	Tournament ($k = 2$)
Elitism Rate	20%
Crossover Rate	60%
Mutation Rate	20%
Mutation Scale	0.5

C. Particle Swarm Optimization (PSO)

1) Algorithm Structure: The optimization process treats the control points of the spline as the particle's position vector $\mathbf{x}_i$ [13]. The swarm is initialized with a population size of $N_{pop} = 20$ and evolves over $N_{gen} = 5$ generations per sector. The velocity update rule governing the particle's movement is given by:

$$\mathbf{v}_i^{k+1} = w^k \mathbf{v}_i^k + c_1 r_1 (\mathbf{p}_{best,i} - \mathbf{x}_i^k) + c_2 r_2 (\mathbf{g}_{best} - \mathbf{x}_i^k) \quad (10)$$

where $c_1 = c_2 = 2.0$ are the cognitive and social acceleration coefficients, and $r_1, r_2 \sim U(0, 1)$ are random stochastic vectors.

2) Adaptive Inertia Weight: To balance global exploration and local exploitation, we employ a linear decay strategy for the inertia weight w. As the generation index g progresses from 1 to N_{gen}, the inertia reduces linearly to decrease exploration:

$$w^g = w_{max} - \frac{g}{N_{gen}}(w_{max} - w_{min}) \quad (11)$$

In our implementation, we set $w_{max} = 0.9$ and $w_{min} = 0.4$, promoting aggressive search in early generations and fine-tuning in later stages.

3) Convergence and Safety: The algorithm tracks the global best cost (g_{best}) across N_{trials} statistical runs to verify robustness. A Death Penalty mechanism is integrated into the fitness evaluation: if the Euclidean distance between the two agents falls below 3.5m, the cost is set to a prohibitively high value (10^4), effectively pruning unsafe trajectories from the solution space immediately.

TABLE III: PSO Hyperparameters

Parameter	Value
Population Size	20
Generations per Sector	5
Inertia Weight (w)	$0.9 \rightarrow 0.4$ (Linear)
Cognitive Coeff. (c_1)	2.0
Social Coeff. (c_2)	2.0
Topology	Global Best

D. Harris Hawks Optimization (HHO)

Harris Hawks Optimization (HHO) is a nature-inspired, population-based meta-heuristic algorithm [14]. It mimics the cooperative behavior and chasing style of Harris' hawks in nature, specifically their surprise pounce strategy. The algorithm models the dynamic interaction between the hawks and the prey through three distinct phases: exploration, transition, and exploitation.

1) Exploration Phase: In the global search phase, the hawks wait to detect the prey. Mathematically, the position of the hawks is updated based on two strategies determined by a random variable q. X_{rand} represents a randomly generated candidate solution vector. The position update is calculated as follows:

$$X(t+1) = \begin{cases} X_{rand}(t) - r_1 |X_{rand}(t) - 2r_2 X(t)|, & \text{if } q \geq 0.5 \\ (X_{rabbit}(t) - X_m(t)) \\ \quad - r_3(LB + r_4(UB - LB)), & \text{if } q < 0.5 \end{cases} \quad (12)$$

where $X(t)$ is the current position vector, $X_{rabbit}(t)$ is the position of the prey (the best solution found so far), $X_m(t)$ is the average position of the current population, and r_1, r_2, r_3, r_4, q are random numbers inside $(0, 1)$. LB and UB represent the lower and upper bounds of the search space, respectively.

2) Transition Phase: The algorithm switches between exploration and exploitation based on the escaping energy of the prey (E). The energy decreases during the iterative process to simulate the prey getting tired:

$$E = 2E_0 \left(1 - \frac{t}{T}\right) \quad (13)$$

where E_0 is the initial energy state randomly selected from $[-1, 1]$, t is the current iteration, and T is the maximum number of iterations. When $|E| \geq 1$, the hawks explore different regions (global search); when $|E| < 1$, they exploit the neighborhood of the solutions (local search).

3) Exploitation Phase: In this local search phase, the hawks perform the surprise pounce. Depending on the prey's remaining energy $|E|$ and the chance of escape r, HHO utilizes four strategies: Soft Besiege, Hard Besiege, Soft Besiege with Progressive Rapid Dives, and Hard Besiege with Progressive Rapid Dives. These strategies allow the algorithm to adaptively converge toward the optimal flight path while avoiding local optima.

E. Deep Reinforcement Learning (PPO)

To overcome the computational latency of iterative metaheuristics, we propose a learning-based framework using Proximal Policy Optimization (PPO). The agent is trained to map track geometry directly to optimal spline control points in a single inference step.

1) Network Architecture: We employ an Actor-Critic architecture with a shared feature extraction backbone.

- **Shared Layers:** The input state is processed by a fully connected (FC) layer with 256 neurons, followed by a ReLU activation.
- **Actor Network:** Branches into a Mean Path (FC-128) and a Standard Deviation Path (FC-128 $\rightarrow$ Softplus), allowing the agent to output a stochastic policy π_θ.
- **Critic Network:** Branches into a Value Path (FC-128) to estimate $V(s)$.

2) Two-Stage Training Protocol: To achieve both robust exploration and high-precision trajectory smoothness, we implemented a two-stage training process:

1) **Stage I (Coarse Learning):** The agent is initialized with random weights and trained with a high learning rate ($\alpha = 1 \times 10^{-4}$) and high entropy regularization ($\beta = 0.02$). This phase encourages the agent to explore the state space and learn basic collision avoidance.
2) **Stage II (Fine-Tuning):** The best performing agent from Stage I is loaded and retrained with a reduced learning rate ($\alpha = 1 \times 10^{-5}$). The entropy coefficient is lowered to $\beta = 0.01$ to reduce stochastic noise. This phase allows the agent to converge on smoother racing lines without the risk of catastrophic forgetting.

3) Reward Function: The reward r is designed to penalize overlap and encourage smoothness. Because Reinforcement Learning optimizes for maximum reward, we defined the RL reward function as the exact negation of the metaheuristic cost function. This transforms our minimization problem into a maximization one while ensuring an equivalent optimization landscape across all evaluated algorithms.

$$r = -\frac{1}{100}\left(w_t T_{lap} + w_w(1 - W_{final}) + w_s \sum \kappa^2\right) - P_{col} \tag{14}$$

where P_{col} is a step penalty for safety violations.

TABLE IV: PPO Training Stages and Hyperparameters

Parameter	**Stage I (Init)**	**Stage II (Fine-Tune)**
Optimizer	Adam	Adam
Learning Rate (α_{PPO})	1×10^{-4}	1×10^{-5}
Entropy Coeff. (β)	0.02	0.01
Clip Factor (ϵ)	0.2	0.2
Mini-Batch Size	64	64
Max Episodes	10,000	10,000
Total Time (m)	298	26
Reward Weights		
Smoothness Penalty (w_s)	500	50,000
tire Wear Weight (w_w)	500	
Lap Time Weight (w_t)	100	
Overlap penalty (w_o)	10	
Collision Penalty (P_{col})	2000	

IV. Experiments and Results

All algorithms were evaluated on the Autodromo Nazionale Monza circuit, with particular focus on the Parabolica corner that amplifies the trade-off between lap time minimization and tire degradation. Convergence plots show the **combined performance of both vehicles**, reporting both Total lap time and Total tire wear.

A. Convergence Behavior of Metaheuristic Algorithms

1) Genetic Algorithm (GA)

Figures 1-3 show the convergence characteristics and resulting racing line for GA. The tire convergence plot shown in Fig.1 shows a gradual decrease across generations, indicating stable exploitation behavior once promising individuals dominate the population. However, convergence is relatively slow. Similarly, the lap time convergence shown in Fig. 2 shows improvement, eventually stabilizing for each vehicle. The resulting trajectory shown in Fig. 3 demonstrates good use of track width but avoids extremely aggressive corner entry.

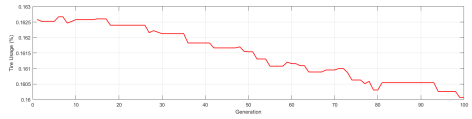

Fig. 1: GA tire Convergence

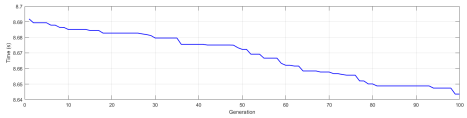

Fig. 2: GA Time Convergence

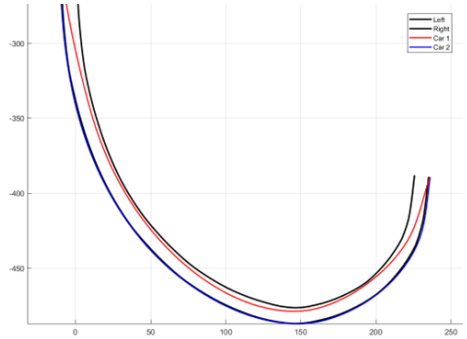

Fig. 3: GA Race Line

2) Particle Swarm Optimization (PSO)

The PSO convergence behavior demonstrates significantly faster improvement compared to GA as shown in Figs 4-5. A sharp reduction in both tire wear and lap time is observed during the first few generations, indicating strong global exploration driven by the high initial inertia weight. As the inertia weight decays linearly, the swarm transitions into a fine exploitation phase, stabilizing rapidly near the optimal solution. The racing line in Fig. 6 aggressively maximizes track width

at corner entry and exit enabling higher exit velocity from the Parabolica corner.

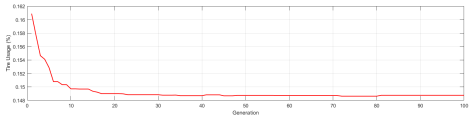

Fig. 4: PSO tire Convergence

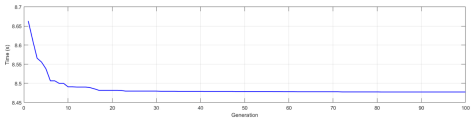

Fig. 5: PSO Time Convergence

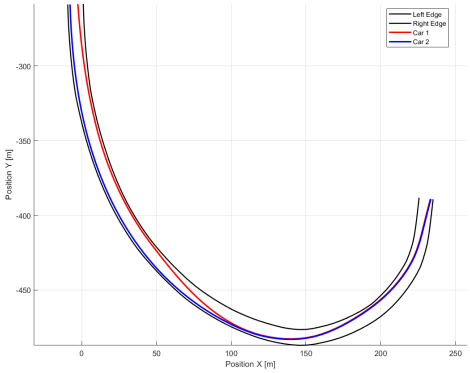

Fig. 6: PSO Race Line

3) Harris Hawks Optimization (HHO)
The convergence curves of HHO shown in Figs. 7 and 8 show an initial rapid improvement phase corresponds to the exploration stage, where hawks search broadly across the solution space. As the escaping energy parameter decreases, the algorithm transitions into exploitation, resulting in smooth stabilization near the optimum. The race line illustrated in Fig. 9 closely resembles the PSO-generated trajectory, indicating convergence toward a similar near-optimal solution. However, HHO achieves slightly improved tire preservation compared to PSO, as shown in Table VI, while maintaining competitive lap times (76.41 ± 0.09 s and 76.56 ± 0.09 s).

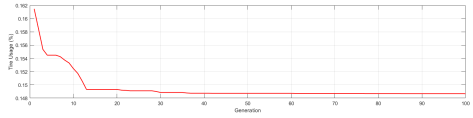

Fig. 7: HHO tire Convergence

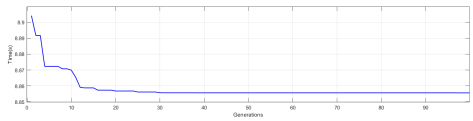

Fig. 8: HHO Time Convergence

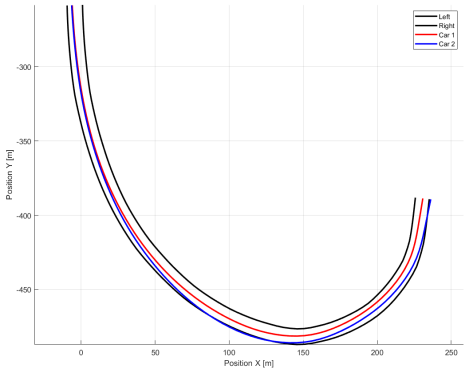

Fig. 9: HHO Race Line

B. Reinforcement Learning (PPO) Performance

Unlike metaheuristic methods, PPO generates a trajectory through a single forward pass after training, eliminating iterative optimization during inference. The race line shown in Fig. 10 appears smoother and more conservative compared to PSO and HHO, prioritizing stable curvature transitions over aggressive corner entry. While formulated to handle multi-vehicle scenarios, the current architecture employs a centralized training scheme. A single policy outputs the joint trajectory offsets for all vehicles simultaneously, treating the coordination task as a high-dimensional single-agent optimization problem.

As reported in Table VI, the RL agent achieves mean lap times of 78.87 s and 79.69 s for vehicles 1 and 2 respectively. While approximately 2.7 seconds slower than PSO, the computational time is drastically reduced to only 14.13 seconds, compared to 866–1455 seconds for metaheuristics. This represents nearly a $60\times$ reduction in computational latency.

The zero standard deviation reflects deterministic inference ensuring repeatable performance without stochastic variability.

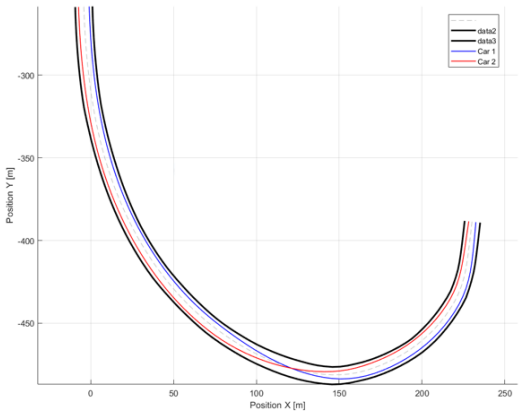

Fig. 10: RL Race Line

TABLE V: Case Study Analysis (RL Agent)

Metric	Spa	Yas Marina	Monza
Total Lap Time(s) (vehicle 1)	99.72	82.86	78.87
Total Lap Time(s) (vehicle 2)	101.22	83.48	79.69
Fastest F1 lap(s) (Qualifying)	101.25	82.207	78.89
Total tire Usage (vehicle 1)	1.19%	0.87%	0.77%
Total tire Usage (vehicle 2)	1.38%	0.88%	0.78%

The cross-track validation results in Table V demonstrate strong generalization capability of the trained PPO agent. The lap times generated by the RL model closely approximate real Formula 1 qualifying benchmarks.

This indicates that the learned policy captures vehicle–track interaction dynamics rather than overfitting to a single circuit. The consistent tire usage percentages across tracks further confirm stable optimization behavior under varying geometries.

To ensure statistical reliability, each algorithm within both the metaheuristic and reinforcement learning frameworks was evaluated across five trials. The performance metrics include mean lap time, mean tire wear, computational time, memory utilization, and standard deviation.

TABLE VI: Performance Statistics (Best vs. Mean $\pm$ Std. Dev)

Metric	SA	GA	PSO	HHO	RL
Computational					
Time (s)	907.11	1454.97	866.11	1145.79	**14.13**
Memory (KB)	2.34	17.19	3.44	3.44	**0.02**
Vehicle 1 Performance					
Best Lap Time (s)	77.35	78.72	**76.17**	76.30	78.87
Mean Lap Time (s)	77.24 ± 0.18	79.02 ± 0.33	76.22 ± 0.04	76.41 ± 0.09	78.87 ± 0.00
Tire Wear (percent)	0.61 ± 0.01	0.72 ± 0.02	0.73 ± 0.01	0.66 ± 0.01	0.77 ± 0.00
Vehicle 2 Performance					
Best Lap Time (s)	77.31	78.95	**76.19**	76.44	79.69
Mean Lap Time (s)	77.11 ± 0.18	79.34 ± 0.35	76.27 ± 0.09	76.56 ± 0.09	79.69 ± 0.00
Tire Wear (percent)	0.59 ± 0.01	0.72 ± 0.02	0.73 ± 0.01	0.69 ± 0.01	0.78 ± 0.00

PSO achieves the fastest lap times but requires significant computation time. HHO provides comparable performance with slightly improved tire preservation. GA demonstrates stable but slower convergence, while SA prioritizes tire longevity at the expense of peak performance. In contrast, PPO sacrifices approximately 3.5% lap time performance relative to the best metaheuristic results but achieves real-time computational feasibility. Metaheuristics define the theoretical performance ceiling, whereas reinforcement learning enables dynamic onboard re-planning suitable for real-time racing applications.

V. Conclusion

This study presented a comparative framework for racing line optimization, benchmarking meta-heuristic algorithms (PSO, HHO, GA, SA) against a Deep Reinforcement Learning (PPO) agent on the Monza circuit. Our results reveal a trade-off between solution optimality and computational latency.

Particle Swarm Optimization (PSO) and Harris Hawks Optimization (HHO) proved to be the superior offline solvers achieving the theoretical best lap times of $76.17s$ and $76.30s$ respectively. These methods effectively exploited the full track width and tire limits , making them ideal for pre-race strategy planning where computation time is not a constraint. HHO outperformed GA likely due to its dynamic escape energy parameter, which balances exploration and exploitation more effectively during the convergence phase. In contrast, the RL agent, while approximately $2.7s$ slower per lap, achieved a reduction in computational time ($14.13s$ vs. $866s$ or more) compared to the meta-heuristics. The 3.5% performance deficit in RL is an acceptable trade-off for the $60\times$ reduction in computational time.

We conclude that while meta-heuristics remain the standard for defining the limit of vehicle performance while PPO offers the necessary speed for real-time onboard trajectory replanning.

References

[1] A. Jain and M. Morari, "Computing the racing line using bayesian optimization," in *2020 59th IEEE Conference on Decision and Control (CDC)*. IEEE, 2020, pp. 6192–6197.

[2] T.-C. Negură, I. Popa, and A. Băicoianu, "Race line optimization with metaheuristics," in *2025 International Symposium on Signals, Circuits and Systems (ISSCS)*. IEEE, 2025, pp. 1–4.

[3] R. E. Aly, Y. A. Abu-Krisha, O. M. Fatehy, A. Y. Ali, C. M. Elias, and O. M. Shehata, "An extended coronavirus optimization algorithm for trajectory planning of a model-based racing track," in *2023 9th International Conference on Control, Decision and Information Technologies (CoDIT)*. IEEE, 2023, pp. 1595–1600.

[4] J. Klapálek, A. Novák, M. Sojka, and Z. Hanzálek, "Car racing line optimization with genetic algorithm using approximate homeomorphism," in *2021 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2021, pp. 601–607.

[5] R. Chen, "The improvements to the minimum curvature race line optimization," in *Journal of Physics: Conference Series*, vol. 2634, no. 1. IOP Publishing, 2023, p. 012002.

[6] R. Rajamani, *Vehicle dynamics and control*. Springer Science & Business Media, 2011.

[7] T. Gillespie, *Fundamentals of Vehicle Dynamics*. SAE International, 2021. [Online]. Available: https://books.google.com.eg/books?id=ZuObEAAAQBAJ

[8] H. B. Pacejka, *Tire characteristics and vehicle handling and stability*. Butterworth-Heinemann Oxford, UK, 2012.

[9] W. J. West and D. Limebeer, "Optimal tyre management for a high-performance race car," *Vehicle system dynamics*, vol. 60, no. 1, pp. 1–19, 2022.

[10] J. Archard, "Contact and rubbing of flat surfaces," *Journal of applied physics*, vol. 24, no. 8, pp. 981–988, 1953.

[11] S. Kirkpatrick, C. D. Gelatt Jr, and M. P. Vecchi, "Optimization by simulated annealing," *science*, vol. 220, no. 4598, pp. 671–680, 1983.

[12] D. Goldberg, *Genetic Algorithms in Search, Optimization, and Machine Learning*. Addison-Wesley, 2002. [Online]. Available: https://books.google.com.eg/books?id=oyB8xgEACAAJ

[13] J. Kennedy and R. Eberhart, "Particle swarm optimization," in *Proceedings of ICNN'95-international conference on neural networks*, vol. 4. ieee, 1995, pp. 1942–1948.

[14] A. A. Heidari, S. Mirjalili, H. Faris, I. Aljarah, M. Mafarja, and H. Chen, "Harris hawks optimization: Algorithm and applications," *Future generation computer systems*, vol. 97, pp. 849–872, 2019.